Bluefish C:\Users\viby\ecen45632_f26\python\CourseDesign\Project1_explore\ClaudeAI_solution\run_all

```python
# from ClaudeAI solution
"""Runs every experiment in the assignment; saves figures, metrics.json/CSV and WAVs to ./results."""
import json, time
from pathlib import Path
import numpy as np
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import signal
from common import FS, fixtures, read_wav, write_wav, relerr, snr, receive
import student_dsp as dsp

SEED = 2735
rng = np.random.default_rng(SEED)
out = Path("results"); out.mkdir(exist_ok=True)
a = fixtures(Path("signals"))
R = {}

def spec_db(X, floor=-80):
    m = np.abs(X); m = m / m.max()
    return np.maximum(20 * np.log10(m + 1e-12), floor)

def show(ax, X, meta, title, fmax=4000):
    f = dsp.frame_freqs(meta); t = dsp.frame_times(meta)
    im = ax.pcolormesh(t, f, spec_db(X).T, vmin=-80, vmax=0, shading="auto")
    ax.set_ylim(0, fmax); ax.set_xlabel("Time (s)"); ax.set_ylabel("Frequency (Hz)")
    ax.set_title(f"{title}\nfs={meta['fs']} Hz, {meta['window']}, L={meta['L']}, H={meta['hop']}, N={meta['N']}", fontsize=8)
    return im

# ---------------------------------------------------------------- A1
p = a["probe"]; A1 = {}
fig, axs = plt.subplots(1, 4, figsize=(18, 4), sharey=True)
for ax, (L, N) in zip(axs, [(256, 256), (1024, 1024), (4096, 4096), (1024, 4096)]):
    X, m = dsp.analysis(p, L=L, hop=L // 4, N=N)
    im = show(ax, X, m, "Relative dB")
    t = dsp.frame_times(m); f = dsp.frame_freqs(m)
    # transient (5 ms Hann pulse) is lowpass: use the 50-250 Hz band, away from tones/chirp, t in 0.7-1.1 s
    win_t = (t > 0.7) & (t < 1.1)
    lo = np.sum(np.abs(X[:, (f > 50) & (f < 250)]) ** 2, axis=1) * win_t
    above = t[lo > 0.1 * lo.max()]                                  # -10 dB frame-energy threshold
    mid = (t > 1.5) & (t < 3.0)
    avg = np.abs(X[mid]).mean(axis=0)
    g = lambda fq: np.interp(fq, f, avg)
    i1, i2 = np.argmin(abs(f - 1000)), np.argmin(abs(f - 1080))     # nearest bins to each tone
    peaks = min(avg[i1], avg[i2]); valley = avg[i1:i2 + 1].min()
    A1[f"L{L}_N{N}"] = dict(Tw_ms=1e3 * L / FS, bin_Hz=FS / N, Th_ms=1e3 * L / 4 / FS,
        transient_width_ms_at_minus10dB=1e3 * (above.max() - above.min()) if len(above) else 0,
        tone_valley_dB_below_peaks=float(20 * np.log10(peaks / valley)),
        chirp_spread_Hz_per_frame=float(750 * L / FS))
fig.colorbar(im, ax=axs, label="dB rel. max"); fig.savefig(out / "A1_spectrograms.png", dpi=110); plt.close(fig)
R["A1"] = A1

# ---------------------------------------------------------------- A2
_, x = read_wav("signals/leakage.wav"); L, N = 1024, 65536
fN = np.fft.rfftfreq(N, 1 / FS); A2 = {}
fig, axs = plt.subplots(1, 2, figsize=(13, 4))
for name in ["rect", "hann", "hamming", "blackman"]:
    w = dsp.make_window(name, L)

```

```
Bluefish C:\Users\viby\ecen45632_f26\python\CourseDesign\Project1_explore\ClaudeAI_solution\run_all
    S = np.abs(np.fft.rfft(x * w, N)) / w.sum() * 2          # coherent-gain corrected amplitude
    axs[0].plot(fN, 20 * np.log10(S + 1e-12), label=name)
    W = np.abs(np.fft.rfft(w, N)); W /= W[0]
    k = 0
    while W[k + 1] < W[k]: k += 1                            # first null = first local minimum
    side = 20 * np.log10(W[k:].max())
    axs[1].plot(fN, 20 * np.log10(W + 1e-12), label=name)
    # leakage of the strong tone alone at 1140 Hz vs. the weak tone's true level
    strong = .65 * np.cos(2 * np.pi * 1003 * np.arange(L) / FS)
    Ss = np.abs(np.fft.rfft(strong * w, N)) / w.sum() * 2
    i = np.argmin(abs(fN - 1140))
    A2[name] = dict(mainlobe_null_to_null_Hz=2 * fN[k], highest_sidelobe_dB=float(side),
        ENBW_Hz=FS * np.sum(w ** 2) / w.sum() ** 2, ENBW_bins=L * np.sum(w ** 2) / w.sum() ** 2,
        strong_tone_leakage_at_1140_dB=float(20 * np.log10(Ss[i] + 1e-12)),
        weak_tone_true_dB=float(20 * np.log10(.015)),
        weak_tone_measured_dB=float(20 * np.log10(S[np.argmax(S * ((fN > 1120) & (fN < 1160)))])))
axs[0].set(xlim=(900, 1300), ylim=(-120, 0), xlabel="Frequency (Hz)", ylabel="Amplitude (dB re 1)",
           title="Two-tone, L=1024, N=65536, fs=16 kHz"); axs[0].legend()
axs[1].set(xlim=(0, 200), ylim=(-120, 0), xlabel="Frequency (Hz)", ylabel="dB re DC", title="Window transforms alone")
fig.savefig(out / "A2_leakage.png", dpi=110); plt.close(fig); R["A2"] = A2

# ---------------------------------------------------------------- B1
B1 = []
for win, ovs in [("hann", [0, .25, .5, .75, .875]), ("rect", [0])]:
    for ov in ovs:
        hop = int(round(1024 * (1 - ov))); row = dict(window=win, overlap=ov, hop=hop)
        try:
            t0 = time.perf_counter(); X, m = dsp.analysis(p, 1024, hop, window=win)
            y, d = dsp.synthesis(X, m); row.update(runtime_ms=1e3 * (time.perf_counter() - t0))
            row.update(frames=X.shape[0], relerr=relerr(p, y), den_min=float(d.min()), den_max=float(d.max()))
        except dsp.NOLAError as e:
            row.update(status="NOLA failure", message=str(e))
        B1.append(row)
write_wav(out / "_tmp.wav", p); _, back = read_wav(out / "_tmp.wav"); (out / "_tmp.wav").unlink()
R["B1"] = dict(table=B1, wav_roundtrip_relerr=relerr(p, back))

# ---------------------------------------------------------------- B2
h = signal.firwin(129, 1200, fs=FS, window="hamming"); ref = np.convolve(a["mixture"], h); B2 = []
for B in [128, 512, 2048]:
    y = dsp.ola_filter(a["mixture"], h, B)
    B2.append(dict(B=B, nfft=1 << int(np.ceil(np.log2(B + 128))), blocks=int(np.ceil(len(a["mixture"]) / B)),
                   relerr=relerr(ref, y), len_ok=len(y) == len(ref)))
# aliasing demo: FFT shorter than B+M-1 circularly wraps the tail
B, nf = 512, 512; blk = a["mixture"][:B]
alias = np.fft.irfft(np.fft.rfft(blk, nf) * np.fft.rfft(h, nf), nf)
B2.append(dict(demo="nfft=B (too short)", relerr_vs_linear_first512=relerr(np.convolve(blk, h)[:nf], alias)))
R["B2"] = B2

# ---------------------------------------------------------------- C1
def evaluate(L, w, stationary=None):
    y, dg = dsp.remove_jammer(a["mixture"], L=L, width_hz=w, stationary_hz=stationary)
    mj = dsp.apply_mask(a["melody"], dg["G"], dg["meta"]); jj = dsp.apply_mask(a["jammer"],
```

```
Bluefish C:\Users\viby\ecen45632_f26\python\CourseDesign\Project1_explore\ClaudeAI_solution\run_all
    dg["G"], dg["meta"])
110     return y, dg, dict(L=L, width_Hz=w, stationary=stationary, snr_in=snr(a["melody"],
    a["mixture"]),
111         snr_out=snr(a["melody"], y), snr_gain=snr(a["melody"], y) - snr(a["melody"],
    a["mixture"]),
112         jammer_atten_dB=float(10 * np.log10(np.sum(a["jammer"] ** 2) / np.sum(jj ** 2))),
113         desired_relerr_pct=100 * relerr(a["melody"], mj))
114 C1 = [evaluate(L, w)[2] for L in (256, 1024, 4096) for w in (30, 60, 120)]
115 C1 += [evaluate(1024, w, 2400.)[2] for w in (30, 60, 120)]
116 y, dg, best = evaluate(1024, 60); R["C1"] = dict(sweep=C1, default=best,
117     ridge_max_err_vs_truth_Hz=float(np.max(np.abs(dg["ridge_hz"] - (1800 + 300 *
    dg["times"]))))))   # evaluation only
118 fig, axs = plt.subplots(1, 3, figsize=(16, 4), sharey=True)
119 for ax, sig_, ttl in zip(axs, [a["melody"], a["mixture"], y], ["Clean", "Jammed",
    "Cleaned"]):
120     X, m = dsp.analysis(sig_, 1024, 256); show(ax, X, m, ttl, fmax=4000)
121 axs[1].plot(dg["times"], dg["ridge_hz"], "w--", lw=1)
122 fig.savefig(out / "C1_spectrograms.png", dpi=110); plt.close(fig)
123 scale = 0.9 / max(np.abs(a["mixture"]).max(), np.abs(y).max())      # one shared scale
124 yf = dsp.ola_filter(a["mixture"], h, 512)[64:64 + len(a["mixture"])]  # remove 64-sample
    group delay
125 for n_, s_ in [("original_mixture", a["mixture"]), ("cleaned_mixture", y),
    ("filtered_audio", yf)]:
126     write_wav(out / f"{n_}.wav", s_, scale=scale)
127
128 # ------------------------------------------------------------------ C2
129 tx = a["tx"]; X, m = dsp.analysis(tx); C2 = {}
130 variants = {"original": tx, "reconstructed": dsp.synthesis(X, m)[0],
131             "magnitude_only": dsp.synthesis(np.abs(X).astype(complex), m)[0],
132             "random_frame_phase": dsp.synthesis(X * np.exp(2j * np.pi *
    rng.random((X.shape[0], 1))), m)[0]}
133 fig, axs = plt.subplots(1, 4, figsize=(16, 4))
134 for ax, (k, v) in zip(axs, variants.items()):
135     met, z = receive(v, a["symbols"], a["bits"], a["pulse"]); met["waveform_relerr"] =
    relerr(tx, v); C2[k] = met
136     ax.plot(z.real, z.imag, ".", ms=2); ax.set_aspect("equal"); ax.set_xlim(-1.5, 1.5);
    ax.set_ylim(-1.5, 1.5)
137     ax.set_title(f"{k}\nEVM {met['evm_percent']:.2f}%  BER {met['ber']:.3f}", fontsize=9);
    ax.set_xlabel("I"); ax.set_ylabel("Q")
138 fig.savefig(out / "C2_constellations.png", dpi=110); plt.close(fig); R["C2"] = C2
139
140 (out / "metrics.json").write_text(json.dumps(R, indent=2, default=float))
141 print(json.dumps(R, indent=1, default=float)[:6000])
142
```